TOP 50 PYTHON INTERVIEW QUESTIONS

@aman_views

1. Why does Python's mutable default argument sometimes remember previous function calls?

→ Default argument values are evaluated once at function definition time. The same object is reused in subsequent calls.

```
def add_item(item, lst=[]):
    lst.append(item)
    return lst

# The same list object is reused
add_item(1)   # [1]
add_item(2)   # [1, 2]
```

2. What is the difference between is and == in Python, and when can they give different results?

→ '==' checks value equality. 'is' checks object identity (i.e., both refer to the same object in memory). They can give different results for distinct objects with the same value.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
a == b    # True  (same values)
a is b    # False (different objects)
a is c    # True  (same object)
```

3. Why can a tuple be used as a dictionary key, but a list cannot?

→ Dictionary keys must be hashable (immutable). A tuple is immutable and hashable (if its elements are hashable), while a list is mutable and therefore unhashable.

```
d = {(1,2): 'point'}   # OK

# d = {[1, 2]: 'point'}  # TypeError
```

4. What is the difference between a list comprehension and a generator expression, and when should you use each?

→ List comprehension creates and returns a list in memory. Generator expression returns a generator (lazy evaluation).
Use list comp. when you need the results multiple times or the dataset is small. Use generator exp. for large datasets or when results are consumed once.

List Comprehension (creates list)

```
squares = [x*x for x in range(5)]
print(squares)  # <class 'list'>
```

Generator Expression (lazy generator)

```
squares_gen = (x*x for x in range(5))
print(squares_gen)   # <class 'generator'>
for val in squares_gen:
    print(val, end=' ')   # One-time iteration
```

5. What happens when you modify a list while iterating over it?

→ Modifying (adding/removing) elements while iterating can lead to skipped elements, repeated elements, or unintended behavior because the iterator index and list size change during iteration.

```
nums = [1, 2, 3, 4, 5]
for n in nums:
    if n % 2 == 0:
        nums.remove(n)   # Madifying list
print(nums)    # Output: [1, 3, 5] (2 and 4 skipped)
```

Safer Approaches:

- Iterate over a copy: for n in nums[:]
- Build a new list
- Iterate in reverse when removing by index

TOP 50 PYTHON INTERVIEW QUESTIONS

6. Why does Python sometimes reuse the same object for small integers or strings?

→ Python uses a mechanism called interning (or caching) for small integers (typically -5 to 256) and some strings to save memory and improve performance. So, identical values in this range may point to the same object in memory.

```
# Small integers are cached
a = 100
b = 100
print(a is b)    # True

# Large integers are not cached
x = 1000
y = 1000
print(x is y)    # False

# String interning (may vary by implementation)
s1 = 'hello'
s2 = 'hello'
print(s1 is s2)  # True (often)
```

7. What is the difference between shallow copy and deep copy in Python?

→ Shallow copy creates a new object but inserts references to the original objects found in the original.

Deep copy creates a new object and recursively copies all objects found in the original.

```
# Shallow copy
import copy
l1 = [[1, 2], [3, 4]]
l2 = copy.copy(l1)
l2[0][0] = 99
print(l1)    # [[99, 2], [3, 4]] (changed)

# Deep copy
l3 = copy.deepcopy(l1)
l3[0][0] = 100
print(l1)    # [[99, 2], [3, 4]] (unchanged)
```

8. What are *args and **kwargs, and how are they different internally?

→ *args collects positional arguments into a tuple.

**kwargs collects keyword arguments into a dictionary.

Internally, *args is a tuple and **kwargs is a dict.

```
def func(*args, **kwargs):
    print('args (tuple):', args)
    print('kwargs (dict):', kwargs)

func(1, 2, 3, name='Aman', age=25)
# Output:
# args (tuple): (1, 2, 3)
# kwargs (dict): {'name': 'Aman', 'age': 25}
```

9. What is a closure in Python, and where would you actually use one?

→ A closure is a function object that remembers values in enclosing scopes even if those scopes are gone. Used in callbacks, decorators, data hiding, and function factories.

```
def outer(x):
    def inner(y):
        return x + y
    return inner

add_five = outer(5)
print(add_five(10))  # 15
# inner() remembers x=5 even after outer() finishes
```

10. What is the difference between an iterator and an iterable in Python?

→ An iterable is any object that can return an iterator (has __iter__ method).

An iterator is an object that yields values one at a time using __next__() and keeps its state.

```
# Iterable: list
lst = [1, 2, 3]
it = iter(lst)    # it is an iterator
print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
```

NOTE:

- Iterable --> can be looped over.
- Iterator ---> keeps track of iteration state.

TOP 50 PYTHON INTERVIEW QUESTIONS

11. What is the difference between a function that returns another function and a function that accepts another function?

→ A function that returns another function is used to create and return a new function (e.g., closures, factory functions).

A function that accepts another function uses that function as an argument (e.g., callbacks, higher-order functions).

```
Returns another function:
def outer(x):
    def inner(y):
        return x + y
    return inner
add_five = outer(5)
print(add_five(10))   # 15
Accepts another function:
def apply_func(func, value):
    return func(value)
def square(n):
    return n * n
print(apply_func(square, 4))   # 16
```

---

12. What are first-class functions in Python?

→ In Python, functions are first-class citizens.

They can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions
- Stored in data structures

```
def greet(name):
    return f"Hello, {name}!"
# Assigned to variable
g = greet
print(g("Aman"))   # Hello, Aman!
# Passed as argument
def call_func(func, n):
    return func(n)
print(call_func(len, [1, 2, 3]))   # 3
# Returned from function
def outer():
    def inner():
        return "I am inner"
    return inner
f = outer()
print(f())   # I am inner
```

---

13. What are decorators in Python, and how do they work internally?

→ Decorators are functions that modify or enhance the behavior of another function without permanently modifying it.

They work internally by taking a function as an argument and returning a new function.

```
# Decorator
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper
@my_decorator
def say_hello():
    print("Hello!")
say_hello()
# Output:
# Before function call
# Hello!
# After function call
```

---

14. How would you create a custom decorator that accepts arguments?

→ Use a decorator factory: an outer function that accepts arguments and returns the actual decorator.

```
def repeat(times):          # decorator factory
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                func(*args, **kwargs)
            return wrapper
    return decorator
@repeat(3)
def greet(name):
    print(f"Hi {name}")
greet("Aman")
                    # Output: Hi Aman (printed 3 times)
```

---

15. Why is functools.wraps used when creating decorators?

→ functools.wraps preserves the metadata of the original function (name, docstring, etc.) when it is wrapped by a decorator.

```
from functools import wraps

def decorator(func):
    @wraps(func)          # preserves metadata
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

TOP 50 PYTHON INTERVIEW QUESTIONS

16. What is the difference between global and nonlocal?

→ global is used to modify a variable in the global (module) scope.
nonlocal is used to modify a variable in the nearest enclosing (non-global) scope.

```
x = 10     # global variable

def outer():
    x = 20     # enclosing scope
    def inner():
        nonlocal x     # refers to outer x
        x = 30
    inner()
    print(x)     # 30

outer()
print(x)     # 10 (global unchanged)
```

17. What is LEGB scope resolution in Python?

→ Python resolves variable names using the LEGB rule:
L - Local (inside current function)
E - Enclosing (in any enclosing functions)
G - Global (module level)
B - Built-in (Python built-in namespace)

```
x = 'global'
def outer():
    x = 'enclosing'
    def inner():
        x = 'local'
        print(x)     # local
    inner()
    print(x)     # enclosing

outer()
print(x)     # global
# Python searches in order: L → E → G → B
```

18. What happens if a variable is defined in both a local and an enclosing scope?

→ The local scope (inside the function) takes precedence over the enclosing scope.
The inner variable shadows the outer one.

```
def outer():
    x = 'enclosing'
    def inner():
        x = 'local'  # shadows enclosing x
        print(x)
    inner()
    print(x)
# Output:
# local
# enclosing
```

19. What is late binding in Python closures?

→ Late binding means that the value of the variable is looked up when the inner function is called, not when it is defined. This can cause unexpected results in loops.

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)
# All lambdas return 2, the last value of i
for f in funcs:
    print(f())     # Output: 2 2 2
# Fix with default argument (early binding):
funcs = []
for i in range(3):
    funcs.append(lambda i=i: i)
for f in funcs:
    print(f())     # Output: 0 1 2
```

20. What is the difference between @staticmethod, @classmethod, and an instance method?

→
- Instance method: works with instance, takes 'self'.
- Class method: works with class, takes 'cls'.
- Static method: no special first parameter.

```
class Demo:
    def instance_method(self):
        return 'instance method'

    @classmethod
    def class_method(cls):
        return 'class method'

    @staticmethod
    def static_method():
        return 'static method'

d = Demo()
print(d.instance_method())     # instance method
print(Demo.class_method())     # class method
print(Demo.static_method())    # static method
```

TOP 50 PYTHON INTERVIEW QUESTIONS

@aman_views

21. What is the difference between `__new__()` and `__init__()`?

→ `__new__()` is a static method used to create a new instance and return it. `__init__()` is an instance method called after the object is created to initialize it.

```
class Demo:
    def __new__(cls, *args, **kwargs):
        print("__new__ called")
        obj = super().__new__(cls)   # creates instance
        return obj
    def __init__(self, name):
        print("__init__ called")
        self.name = name

d = Demo("Aman")

# Output:
# __new__ called
# __init__ called
```

---

22. What is method resolution order (MRO) in Python?

→ MRO is the order in which base classes are searched when calling a method in a derived class. Python uses C3 linearization to determine this order.

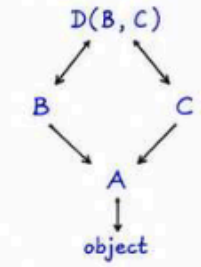
```
class A:
    def show(self): print("A")
class B(A):
    def show(self): print("B")
class C(A):
    def show(self): print("C")
class D(B, C):
    pass
print(D.mro())       # [D, B, C, A, object]
d = D(); d.show()    # Output: B
```

---

23. How does multiple inheritance work in Python?

→ Python supports multiple inheritance. When a method is called on a derived class, Python follows the MRO to find the first occurrence of that method in the base classes and executes it.

```
class A:
    def show(self): print("A")
class B(A):
    def show(self): print("B")
class C(A):
    def show(self): print("C")
class D(B, C):
    pass
d = D()     # MRO: [D, B, C, A, object]
d.show()    # Output: B
```



Search Order: D → B → C → A → object

---

24. What is the C3 linearization algorithm used for in Python?

→ C3 linearization is used to compute the MRO. It preserves the local precedence order and is monotonic, ensuring consistent and predictable inheritance resolution.

Example: For D(B, C) where B(A), C(A)

MRO Calculation:

1. L[D] = [D] + merge(L[B], L[C], [B, C])
2. L[B] = [B, A, object]
3. L[C] = [C, A, object]
4. merge([B, A, object], [C, A, object], [B, C])
   = [B, C, A, object]
5. So, L[D] = [D] + [B, C, A, object]
   = [D, B, C, A, object]

Properties:

- Preserves local order
- Monotonic
- Ensures consistent MRO

---

25. What are magic/dunder methods in Python? Give some real use cases.

→ Magic methods (dunder methods) are special methods with double underscores. They allow customization of built-in operations. They are called implicitly by Python.

Common magic methods & use cases:

```
• __init__(self)          # Constructor
• __str__(self)           # User-friendly string representation
• __repr__(self)          # Official string representation (debugging)
• __len__(self)           # len(object)
• __add__(self, other)    # object1 + object2
• __eq__(self, other)     # object1 == object2
• __enter__/__exit__      # Context management (with statement)
• __getitem__(self, key)  # Indexing: obj[key]
• __call__(self, ...)     # Makes object callable like a function
```

TOP 50 PYTHON INTERVIEW QUESTIONS

@aman_views

26. What is the difference between `__str__()` and `__repr__()`?

→ `__str__()` is meant for end users and should return a readable string.
`__repr__()` is meant for developers and should return an unambiguous string that represents the object.

```
class User:
  def __init__(self, name, age):
      self.name = name
      self.age = age

  def __str__(self):
      return f"User({self.name}, {self.age})"      # readable

  def __repr__(self):
      return f"User(name='{self.name}', age={self.age})"
                                                   # unambiguous
u = User('Aman', 25)
print(str(u))      # User(Aman, 25)
print(repr(u))     # User(name='Aman', age=25)
```

27. What are properties in Python, and why would you use @property instead of a normal attribute?

→ Properties allow you to define getter, setter, and deleter methods for an attribute.
@property lets you access methods like attributes, while adding validation or computed logic.

```
class Rectangle:
  def __init__(self, width, height):
      self._width = width
      self._height = height

@property
def area(self):
    return self._width * self._height

@area.setter
def area(self, value):
    self._width = value / self._height

r = Rectangle(10, 5)
print(r.area)          # 50
r.area = 100           # calls setter, sets width = 20
print(r.area)          # 100
```

28. What are descriptors in Python, and how do they work?

→ Descriptors are objects that define how attribute access is handled.
They implement `__get__()`, `__set__()`, and/or `__delete__()`. They are the foundation of properties, methods, and staticmethods.

```
# Example of a simple descriptor
class Positive:
   def __set_name__(self, owner, name):
       self.name = name

   def __get__(self, instance, owner):
       return instance.__dict__[self.name]

   def __set__(self, instance, value):
       if value <= 0:
           raise ValueError("Must be positive")
       instance.__dict__[self.name] = value
class Product:
   price = Positive()    # descriptor in action
p = Product()
p.price = 100         # OK
print(p.price)        # 100
# p.price = -5        # ValueError
```

29. What is `__slots__`, and when can it improve a Python class?

→ `__slots__` limits the set of attributes that can be created for instances.
It saves memory by preventing the creation of `__dict__` for each instance.
Useful when you have many objects with a fixed set of attributes.

```
class Point:
  __slots__ = ('x', 'y')      # no __dict__ created

 def __init__(self, x, y):
     self.x = x
     self.y = y

p = Point(1, 2)
# p.z = 5        # AttributeError: 'Point' object has no attribute 'z'
```

30. What is duck typing in Python, and how is it different from traditional type checking?

→ Duck typing means "If it walks like a duck and quacks like a duck, it is a duck."
Python focuses on behavior, not type.
Traditional type checking relies on explicit types or inheritance.

```
# Duck typing example
class Duck:
   def quack(self):
       return "Quack!"
class ToyDuck:
   def quack(self):
       return "Squeak!"

def make_it_quack(obj):
    print(obj.quack())          # Works if object has quack()
make_it_quack(Duck())           # Quack!
make_it_quack(ToyDuck())        # Squeak!
```

TOP 50 PYTHON INTERVIEW QUESTIONS

31. What is the difference between yield and return?

→ return exits the function and sends a value back to the caller.

yield pauses the function and returns a value, but remembers the state for next time.

```
# Using return
def get_list():
    return [1, 2, 3]

# Using yield
def gen():
    yield 1
    yield 2
    yield 3
for val in gen():
    print(val)    # 1  2  3
```

32. How do generators save memory compared with lists?

→ Generators produce values one at a time on demand and do not store all values in memory at once.

Lists store all values in memory.

So, generators are memory efficient.

```
# List (stores all values in memory)
my_list = [i for i in range(1000000)]
# Uses more memory

# Generator (produces one by one)
my_gen = (i for i in range(1000000))
# Uses less memory

import sys
print(sys.getsizeof(my_list))    # large size
print(sys.getsizeof(my_gen))     # small size
```

33. What happens internally when a generator function is called?

→ When a generator function is called, it does not execute the body immediately.

Instead, it returns a generator object.

The code runs only when next() is called on the generator.

```
def simple_gen():
    print("Start")
    yield 1
    print("Middle")
    yield 2
    print("End")

g = simple_gen()   # function called, nothing printed
print(next(g))     # Output: Start  (then 1)
print(next(g))     # Output: Middle (then 2)
# next(g) again -> prints End, then StopIteration
```

34. What is the difference between a generator, iterator, and iterable?

→
- Iterable: Any object that can return an iterator using __iter__().
- Iterator: An object that has __next__() and __iter__() and returns values one at a time.
- Generator: A special type of iterator created using a function with yield.

```
# Iterable example
lst = [1, 2, 3]      # list is iterable
it = iter(lst)        # gets iterator
print(next(it))       # 1
print(next(it))       # 2
# Generator is also iterator
def gen():
    yield 1
    yield 2
    yield 3
g = gen()             # generator object (iterator)
print(next(g))        # 1
print(next(g))        # 2
```

35. How does Python manage memory?

→ Python manages memory automatically.

It uses a private heap to store objects and frees memory when objects are no longer referenced. It uses reference counting and a garbage collector to reclaim unused memory.

```
a = [1, 2, 3]         # object created
b = a                 # reference count +1
del a                 # reference count -1
# when ref count becomes 0 -> object deleted

import gc
gc.collect()          # manually trigger GC
# GC removes objects in cycles
```

TOP 50 PYTHON INTERVIEW QUESTIONS

36. What is reference counting in Python?

→ Reference counting is a memory management technique where Python keeps track of the number of references to an object.
When the reference count becomes 0, the object is deallocated (memory is freed).

```
# Reference counting example
import sys
a = [1, 2, 3]
print(sys.getrefcount(a))   # 2 (a + getrefcount arg)
b = a
print(sys.getrefcount(a))   # 3 (a, b + arg)
del b
print(sys.getrefcount(a))   # 2 (only a + arg)
del a
# Now reference count = 0 → object is freed
```

37. How does Python's garbage collector handle cyclic references?

→ Reference counting alone cannot free cyclic references (objects referring to each other).
Python's garbage collector (GC) detects unreachable cycles and frees the memory using a generational collection algorithm.

```
# Example: cyclic reference
class Node:
    def __init__(self):
        self.ref = None
a = Node()
b = Node()
a.ref = b
b.ref = a

# a and b refer to each other → reference count never 0
# But GC detects unreachable cycle and collects them.
# You can manually trigger GC:
import gc
gc.collect()
```

38. What is the difference between del, clear(), and assigning None?

→
- del : Deletes the variable name (reference) completely.
- clear(): Empties the container in-place (object still exists).
- None : Rebinds the variable to None object (old object becomes eligible for GC if no other references).

```
# Examples
lst = [1, 2, 3]
# clear() → empties the list
lst.clear()
print(lst)        # []
lst = [1, 2, 3]
del lst           # deletes the variable lst
# print(lst)      # NameError: name 'lst' is not defined

lst = [1, 2, 3]
lst = None        # lst now refers to None
print(lst)        # None
```

39. What is the difference between raise, raise from, and re-raising an exception?

→
- raise e : Raises an exception e.
- raise New from e: Raises New with the original exception e as the cause.
- raise : Re-raises the current exception (preserves the traceback).

```
# raise
  raise ValueError("Invalid value")
# raise from
try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Conversion failed") from e
# Re-raise (inside except block)
try:
    int("abc")
except ValueError:
    print("Logging error...")
    raise       # re-raises the same ValueError
```

40. What are context managers, and how would you create a custom context manager?

→ Context managers help manage resources (like files, DB connections) safely.
They ensure setup before a block and cleanup after the block, even if an exception occurs.
We create them using the contextmanager decorator or by implementing __enter__() and __exit__().

```
# Using contextmanager decorator
from contextlib import contextmanager
@contextmanager
def open_file(path, mode):
    f = open(path, mode)
    try:
        yield f           # provides the resource
    finally:
        f.close()         # cleanup

# Usage
with open_file("sample.txt", "r") as file:
    data = file.read()
    print(data)
```

TOP 50 PYTHON INTERVIEW QUESTIONS

@aman_views

41. What is the Global Interpreter Lock (GIL), and why does it matter?

→ The GIL is a mutex that allows only one thread to execute Python bytecode at a time, even on multi-core systems.
It matters because it can limit the performance of CPU-bound multi-threaded programs in CPython.

```
# GIL Example (conceptual)
import threading, time
def cpu_task():
    x = 0
    for _ in range(10_000_000):
        x += 1
    print("done")
t1 = threading.Thread(target=cpu_task)
t2 = threading.Thread(target=cpu_task)
t1.start(); t2.start()
t1.join(); t2.join()
# Even with 2 threads, CPU-bound code won't run truly in parallel
# due to GIL in CPython.
```

42. If Python has the GIL, why can multithreading still be useful?

→ Multithreading is still useful for I/O-bound tasks (like file I/O, network calls, DB queries) because while one thread is waiting for I/O, the GIL is released, allowing other threads to run.

```
# I/O-bound example
import threading, time
def io_task(n):
    print(f"Task {n} start")
    time.sleep(2)    # Simulate I/O wait
    print(f"Task {n} end")
threads = [threading.Thread(target=io_task, args=(i,)) for i in range(3)]
start = time.time()
for t in threads: t.start()
for t in threads: t.join()
print("Total time:", round(time.time() - start, 2))  # ~2s, not 6s
```

43. What is the difference between threading, multiprocessing, and asyncio?

→
- Threading: Uses threads (same memory space), good for I/O-bound tasks.
- Multiprocessing: Uses multiple processes (separate memory space), good for CPU-bound tasks, bypasses GIL.
- asyncio: Single-threaded concurrency using async/await, best for high-concurrency I/O-bound tasks.

Model	Memory Space	GIL Impact	Best For
Threading	Shared	Affected	I/O-bound
Multiprocessing	Separate	Bypassed	CPU-bound
asyncio	Shared	N/A	High-concurrency I/O-bound

44. When would you choose multiprocessing instead of multithreading?

→ Choose multiprocessing for CPU-bound tasks that require true parallelism across multiple cores, as it bypasses the GIL.
Each process has its own memory, avoiding GIL contention.

```
# CPU-bound example using multiprocessing
from multiprocessing import Process
def cpu_task():
    x = 0
    for _ in range(10_000_000):
        x += 1
    print("done")
if __name__ == "__main__":
    p1 = Process(target=cpu_task)
    p2 = Process(target=cpu_task)
    p1.start(); p2.start()
    p1.join(); p2.join()   # Runs in parallel on multiple cores
```

45. What is asyncio, and when should you use asynchronous programming?

→ asyncio is Python's library for writing concurrent code using async/await syntax.
Use asynchronous programming for high-concurrency I/O-bound workloads like web servers, HTTP requests, WebSockets, etc.

```
# asyncio example
import asyncio
async def fetch(n):
    print(f"Start {n}")
    await asyncio.sleep(1)    # Simulate I/O wait
    print(f"End {n}")
async def main():
    await asyncio.gather(*(fetch(i) for i in range(3)))
asyncio.run(main())   # All run concurrently (~1s total)
```

TOP 50 PYTHON INTERVIEW QUESTIONS

@aman_views

46. What is a Python decorator, and how does it work?

→ A decorator is a function that takes another function and extends its behavior without modifying the original function. Decorators use the @ syntax for clean and reusable code.

```
# Example: timing decorator
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"Time taken: {end - start:.6f} sec")
        return result
    return wrapper

@timer
def add(a, b):
    time.sleep(1)      # simulate work
    return a + b

add(5, 7)              # Output: Time taken: ~1.000000 sec
```

47. What are lambda functions, and where are they used?

→ Lambda functions are small, anonymous functions defined using the lambda keyword. They are used when a simple function is needed for a short period of time, mostly with map(), filter(), sorted(), reduce() etc.

```
# Examples
add = lambda a, b: a + b          # lambda with expr
print(add(3, 4))                  # 7

nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))    # [1, 4, 9, 16]
evens = list(filter(lambda x: x%2 == 0, nums)) # [2, 4]
sorted_nums = sorted(nums, key=lambda x: -x)  # [4, 3, 2, 1]
```

48. What is the difference between list comprehension and map()?

→ List comprehension is more readable and flexible.
map() returns an iterator (in Python 3) and is better for applying an existing function to every item.

```
# Using list comprehension
nums = [1, 2, 3, 4]
squares_lc = [x*x for x in nums]    # [1, 4, 9, 16]
---------------------------------------
# Using map()
def square(x):
    return x*x
squares_map = list(map(square, nums))    # [1, 4, 9, 16]
# or with lambda
squares_map2 = list(map(lambda x: x*x, nums)) # [1, 4, 9, 16]
```

49. Explain Python's any() and all() functions with examples.

→
- any(iterable) returns True if at least one element is True.
- all(iterable) returns True if all elements are True.

```
# any() example
a = [False, False, True]
print(any(a))              # True
print(any([0, "", None]))  # False

# all() example
b = [True, True, True]
print(all(b))              # True
print(all([1, "a", 3]))    # True
print(all([1, 0, 3]))      # False
```

50. How does Python handle mutable and immutable objects?

→ Immutable objects (int, str, tuple, float) cannot be changed after creation.
Mutable objects (list, dict, set) can be modified in-place.
Python passes references to objects; mutable objects reflect changes.

```
# Immutable example
x = 10
y = x
y = 20        # changes x, y still 10
print(x, y)   # 20 10

# Mutable example
lst1 = [1, 2, 3]
lst2 = lst1
lst1.append(4)    # modifies original list
print(lst1)       # [1, 2, 3, 4]
print(lst2)       # [1, 2, 3, 4]
```